



UNIMORE
UNIVERSITÀ DEGLI STUDI DI
MODENA E REGGIO EMILIA

Dipartimento di Scienze Fisiche,
Informatiche e Matematiche

13. Assignment 4: Loop Fusion

Linguaggi e Compilatori [I215-011]

Corso di Laurea in INFORMATICA
(D.M.270/04) [16-262]
Anno accademico 2023/2023

Prof. Andrea Marongiu
andrea.marongiu@unimore.it

Copyright note

È vietata la copia e la riproduzione dei contenuti e immagini in qualsiasi forma.

È inoltre vietata la redistribuzione e la pubblicazione dei contenuti e immagini non autorizzata espressamente dall'autore o dall'Università di Modena e Reggio Emilia.

Credits

- Barton, IBM Canada, “Revisiting Loop Fusion and its place in the loop transformation framework”
- Absar, ARM L.t.d., “Scalar Evolution Demystified”

Struttura del passo

- Ci serve un `FUNCTION_PASS`, tramite il quale accedere alle informazioni sui loop

```
PreservedAnalyses LoopFusionPass::run(Function &F,  
                                       FunctionAnalysisManager &AM) {  
    LoopInfo &LI = AM.getResult<LoopAnalysis>(F);  
    for (auto *L : *LI)  
        ...  
}
```

Struttura del passo

- Manipolazione dei Loop
(esempio dal passo LoopVersioning)

```
// Build up a worklist of inner-loops to version. This is necessary as the
// act of versioning a loop creates new loops and can invalidate iterators
// across the loops.
SmallVector<Loop *, 8> Worklist;

for (Loop *TopLevelLoop : *LI)
    for (Loop *L : depth_first(TopLevelLoop))
        // We only handle inner-most loops.
        if (L->isInnermost())
            Worklist.push_back(L);
```

Struttura del passo

- Esplorare i metodi delle classi LoopInfo e LoopInfoBase

Loop Fusion

In order for two loops, L_j and L_k to be fused, they must satisfy the following conditions:

1. L_j and L_k must be adjacent

- There cannot be any statements that execute between the end of L_j and the beginning of L_k*

2. L_j and L_k must iterate the same number of times

3. L_j and L_k must be control flow equivalent

- When L_j executes L_k also executes or when L_k executes L_j also executes*

4. There cannot be any negative distance dependencies between L_j and L_k

- A negative distance dependence occurs between L_j and L_k , L_j before L_k , when at iteration m L_k uses a value that is computed by L_j at a future iteration $m+n$ (where $n > 0$).*

Loop Fusion

In order for two loops, L_j and L_k to be fused, they must satisfy the following conditions:

1. L_j and L_k must be adjacent

- *There cannot be any statements that execute between the end of L_j and the beginning of L_k*

2. L_j and L_k must iterate the same number of times

3. L_j and L_k must be control flow equivalent

- *When L_j executes L_k also executes or when L_k executes L_j also executes*

4. There cannot be any negative distance dependencies between L_j and L_k

- *A negative distance dependence occurs between L_j and L_k , L_j before L_k , when at iteration m L_k uses a value that is computed by L_j at a future iteration $m+n$ (where $n > 0$).*

1) Adiacenza dei loop

Due loop L0 e L1 sono adiacenti se non ci sono basic blocks aggiuntivi nel CFG tra l'uscita di L0 e l'entry di L1.

Se i loop sono **guarded** il successore non loop del guard branch di L0 deve essere l'entry block di L1.

Se i loop **non sono guarded** l'exit block di L0 deve essere il preheader di L1

<https://llvm.org/docs/LoopTerminology.html>

Loop Fusion

In order for two loops, L_j and L_k to be fused, they must satisfy the following conditions:

1. *L_j and L_k must be adjacent*

- *There cannot be any statements that execute between the end of L_j and the beginning of L_k*

2. *L_j and L_k must iterate the same number of times*

3. *L_j and L_k must be control flow equivalent*

- *When L_j executes L_k also executes or when L_k executes L_j also executes*

4. *There cannot be any negative distance dependencies between L_j and L_k*

- *A negative distance dependence occurs between L_j and L_k , L_j before L_k , when at iteration m L_k uses a value that is computed by L_j at a future iteration $m+n$ (where $n > 0$).*

3) Control flow equivalence

- Due loop L0 e L1 si dicono control flow equivalent se quando uno esegue è garantito che esegua anche l'altro.
- Per determinare la control flow equivalenza ci servono le informazioni di dominanza e postdominanza
 - Se L0 domina L1 e L1 postdomina L0 allora i due loop sono control flow equivalenti

Analisi di dominanza e postdominanza

```
PreservedAnalyses LoopFusePass::run(Function &F,  
FunctionAnalysisManager &AM) {  
    DominatorTree &DT =  
        AM.getResult<DominatorTreeAnalysis>(F);  
    PostDominatorTree &PDT =  
        AM.getResult<PostDominatorTreeAnalysis>(F);  
    ...  
}
```